Project Deliverable & Phase-1 of Architecture Definition and Design

Phase 1: Architecture Definition and Low-Power Design of the LPN-1 Microprocessor

Nithin Charan Boppana

Computer Architecture and Design (MSCS-531-M40)

11/6/2026

Phase 1: Architecture Definition and Low-Power Design of the LPN-1 Microprocessor

This report defines LPN-1, a low-power microprocessor for battery-powered wireless sensor nodes and explains the choices behind it. Later phases will implement and evaluate the design in gem5. Energy, not peak speed, is the main constraint for this class of device.

Target Application and Workload

LPN-1 is aimed at an IoT sensor node (say an environmental or industrial monitor) which is going to be running for months on a very small battery or on harvested energy. The workload I'm assuming is that it wakes up on a timer, takes a reading from the sensors, does some fixed-point filtering and feature extraction, packs up a message and goes back to sleep. I'm also assuming it will be active only a very small percentage of the time. This duty cycle is relevant as Phase 2 will be testing with simulated active and idle times. It's important because a core which is mostly sleeping will in fact lose more power to leakage than it will spend on computation.

Key Architectural Features

Table 1 gives the design and Figure 1 shows the power domains. The instruction set is RV32IMC (32-bit integer base with multiply and divide, and compressed instructions). The compressed instructions reduce program size by about 25% and reduce the number of instruction bits fetched by about 25% (Waterman, 2011) which is significant given that instruction fetch is a major component of processor energy. I've omitted the floating-point hardware as the sensor filters can be implemented using fixed point arithmetic.

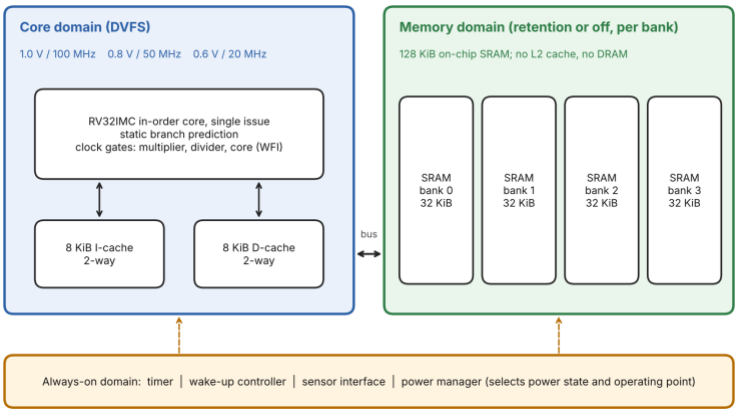
The pipeline is single issue, in-order with static prediction and no branch target buffer (so no predictor storage to leak whilst the core is sleeping). The trade-off in the instruction set is evident in the open-source cores. The small Ibex core (RV32IMC) configuration achieves 2.47 CoreMark per MHz at around 26.6kGE, the smaller RV32EC configuration achieves 0.904 CoreMark per MHz at 16.85kGE (lowRISC, n.d.). That's about 2.7x the performance for 1.6x the area so LPN-1 retains the full RV32IMC set.

Memory is all on chip. A DRAM access is around 1-2 nJ whereas an on-chip cache access or operation is around 10 pJ (Horowitz, 2014) so keeping the working set on chip eliminates the most expensive accesses.

Table 1
LPN-1 Architecture Summary

Feature	Specification
Target use	Battery-powered or energy-harvesting wireless sensor node
Instruction set	RV32IMC (32-bit integer, multiply/divide, compressed); no floating-point unit
Pipeline	In-order, single-issue, short pipeline; static branch prediction, no branch-target buffer
Caches	8 KiB L1 instruction cache and 8 KiB L1 data cache, both 2-way; no L2 cache
Memory	128 KiB on-chip SRAM in four 32 KiB banks; no external DRAM
Operating points	1.0 V at 100 MHz; 0.8 V at 50 MHz; 0.6 V at 20 MHz (assumed values)
Clock gating	Multiplier, divider, and cache arrays when idle; whole core on wait-for-interrupt
Power gating	Per-bank SRAM retention or off; core off in deep sleep
Always-on domain	Timer, wake-up controller, sensor interface, power manager

Figure 1
LPN-1 Block Diagram and Power Domains



Note. The always-on domain controls the power state of the other two domains. Operating points are assumed values.

Low-Power Design Techniques

Dynamic voltage and frequency scaling (DVFS) - dynamic power is proportional to capacitance, the square of the voltage and the frequency (Hennessy & Patterson, 2019). Relative to the top operating point, dynamic power is 32% at 0.8V and 50MHz and 7.2% at 0.6V and 20MHz.

Dynamic energy per cycle is 64% and 36% respectively. I will stop at 0.6V rather than go all the way to near threshold. Dreslinski et al (2010) show massive energy savings at near threshold voltages, but at the expense of lower performance, greater variability and more failures. That's not suitable for a node with sampling deadlines. A lower voltage also increases the active time, so leakage lasts longer. The policy will pick the lowest point that meets the deadline and Phase 2 will compare that against running fast and sleeping at the same time.

Clock gating. Clock gating removes dynamic power from idle logic (Machetti et al., 2024) but leakage power is still present. LPN-1 gates the multiplier, divider and cache arrays when idle and the entire core when it's waiting for an interrupt.

Power gating. Leakage is a major contributor to power consumption in deep sub-micron CMOS (Roy et al. 2003) so clock gating by itself is not enough during long sleeps. LPN-1 powers off the core in deep sleep and leaves only the always-on domain active. It has four memory banks which can be individually retained or powered off. Machetti et al. (2024) report that putting the memory banks into retention reduced their leakage by approximately 42.5% compared to the leakage that would have occurred if the banks were in active state on the chip. Power gating has wake-up energy and latency so the power manager will only enter deep sleep when the expected idle time is greater than the break-even time calculated in Phase 2. Table 2 shows the mapping of the power states to the states defined by gem5.

Table 2

LPN-1 Power States and gem5 Power States

LPN-1 state	Description	gem5 power state
Active	Core executes at one of three operating points	ON
Idle	Core waits for an interrupt; clock stopped	CLK_GATED
Sleep	Core off; SRAM banks with live data retained	OFF (core), SRAM_RETENTION (memory)
Deep sleep	Everything off except the always-on domain	OFF

Justification of Design Choices

The three findings that inform the design are: firstly, Horowitz (2014) reports circa 70 pJ of overhead per instruction as opposed to a few pJ for the actual operation, so the design is focused on the instruction supply side of things (compressed code, small caches, short pipeline, no BTB). Secondly, an idle node is still consuming power through memory leakage and wakeups as well as

computation, which is why LPN-1 uses banked memory, retention and an always-on power manager as X-HEEP does (Machetti et al., 2024). Thirdly, instruction set extensions are optional. Gautschi et al. (2017) added hardware loops, packed SIMD and dot-product instructions to a four-stage RISC-V core and reported on average 3.5x speed and 3.2x better energy efficiency on sensor processing workloads. I treat such extensions as candidates for the optimization phase as gem5 doesn't report their area cost. The main costs of the baseline are that the sensor code is all fixed point and that deep sleep introduces wake-up latency.

Plan for Implementation and Evaluation in gem5

gem5 provides all the pieces that this design requires: voltage and clock domains with lists of operating points, a DVFS handler and power models with ON, CLK_GATED, SRAM_RETENTION and OFF states (gem5 Project, n.d.; Lowe-Power et al., 2020). The documentation that I've read talks about these in the context of ARM examples and a very detailed CPU timing model, but it doesn't say anything about how you switch operating points whilst a program is running. In Phase 2 I'll check on my installed gem5 v25.1.0.1 RISC-V build whether these features work with an in-order CPU model and will amend the plan if they don't. The power figures from gem5 will be modelled estimates based on published data, not actual silicon measurements.

Phase 2 will run the sensor kernels (fixed point filter, transform and threshold detector) and compare the designs based on simulated execution time and energy per task against a baseline with a single operating point and no gating.

References

Dreslinski, R. G., Wieckowski, M., Blaauw, D., Sylvester, D., & Mudge, T. (2010). Near-threshold computing: Reclaiming Moore's law through energy efficient integrated circuits. *Proceedings of the IEEE, 98*(2), 253–266. https://doi.org/10.1109/JPROC.2009.2034764

Gautschi, M., Schiavone, P. D., Traber, A., Loi, I., Pullini, A., Rossi, D., Flamand, E., Gürkaynak, F. K., & Benini, L. (2017). Near-threshold RISC-V core with DSP extensions for scalable IoT endpoint devices. *IEEE Transactions on Very Large-Scale Integration (VLSI) Systems, 25*(10), 2700–2713. https://doi.org/10.1109/TVLSI.2017.2654506

gem5 Project. (n.d.). *gem5 documentation*. https://www.gem5.org/documentation/

Hennessy, J. L., & Patterson, D. A. (2019). *Computer architecture: A quantitative approach* (6th ed.). Morgan Kaufmann.

Horowitz, M. (2014). Computing's energy problem (and what we can do about it). In *2014 IEEE International Solid-State Circuits Conference digest of technical papers* (pp. 10–14). IEEE. https://doi.org/10.1109/ISSCC.2014.6757323

Lowe-Power, J., Ahmad, A. M., Akram, A., Alian, M., Amslinger, R., Andreozzi, M., Armejach, A., Asmussen, N., Beckmann, B., Bharadwaj, S., Black, G., Bloom, G., Bruce, B. R., Carvalho, D. R., Castrillon, J., Chen, L., Derumigny, N., Diestelhorst, S., Elsasser, W., … Zulian, E. (2020). *The gem5 simulator: Version 20.0+* (arXiv:2007.03152). arXiv. https://arxiv.org/abs/2007.03152

lowRISC. (n.d.). *Ibex RISC-V core* [Source code]. GitHub. https://github.com/lowRISC/ibex

Machetti, S., Schiavone, P. D., Müller, T. C., Peón-Quirós, M., & Atienza, D. (2024). *X-HEEP: An open-source, configurable and extendible RISC-V microcontroller for the exploration of ultra-low-power edge accelerators* (arXiv:2401.05548). arXiv. https://arxiv.org/abs/2401.05548

Roy, K., Mukhopadhyay, S., & Mahmoodi-Meimand, H. (2003). Leakage current mechanisms and leakage reduction techniques in deep-submicrometer CMOS circuits. *Proceedings of the IEEE, 91*(2), 305–327. https://doi.org/10.1109/JPROC.2002.808156

Waterman, A. (2011). *Improving energy efficiency and reducing code size with RISC-V compressed* (Technical Report No. UCB/EECS-2011-63). University of California, Berkeley. https://www2.eecs.berkeley.edu/Pubs/TechRpts/2011/EECS-2011-63.html